

---

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Operating Systems — Assignment 2

# Device Drivers & OS Hardware Abstraction

A Technical Report on I/O Request Lifecycle,  
Driver Architecture, and Kernel Safety

**Submitted By:** [Your Full Name]

**Enrollment No.:** [Your Enrollment / Roll Number]

**Course:** B.Tech / M.Tech — Computer Science

**Subject:** Operating Systems

**Subject Code:** [Subject Code]

**Semester:** [Semester] — [Academic Year]

**Submitted To:** [Professor's Name]

**Institution:** [Your College / University Name]

**Date of Submission:** April 26, 2026

---

*"The operating system is the invisible hand that turns hardware into possibility."*

# Abstract

## Report Summary

This technical report presents a comprehensive examination of how modern Operating Systems abstract hardware complexity through the device driver subsystem. Specifically, it addresses four critical aspects: **(a)** the architectural split of a device driver into its Top Half and Bottom Half; **(b)** the driver registration lifecycle including static compilation and dynamic kernel module (LKM) loading; **(c)** a comparative case study between a character-based keyboard driver and a block-based SSD driver, detailing their distinct kernel data structures; and **(d)** error propagation, Kernel Panics, Blue Screens of Death, and the microkernel approach (Minix) to fault isolation.

The report is written from the perspective of a firmware engineer who must understand *exactly which interfaces* their hardware code must conform to in order to be accepted by the Linux kernel. Theoretical analysis is supported by pseudocode, architectural diagrams, comparison tables, and documented real-world behaviour.

**Keywords:** Device Driver, Interrupt Handling, Top Half, Bottom Half, Linux Kernel Module (LKM), `module_init`, `module_exit`, Character Device, Block Device, `file_operations`, `request_queue`, Kernel Panic, BSOD, Microkernel, Minix, I/O Request Lifecycle.

# Contents

|                                                                                                 |           |
|-------------------------------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                                                 | <b>1</b>  |
| <b>List of Figures</b>                                                                          | <b>4</b>  |
| <b>List of Tables</b>                                                                           | <b>5</b>  |
| <b>1 Introduction</b>                                                                           | <b>6</b>  |
| 1.1 Background and Motivation . . . . .                                                         | 6         |
| 1.2 Scope of This Report . . . . .                                                              | 6         |
| <b>2 The Anatomy of a Device Driver</b>                                                         | <b>7</b>  |
| 2.1 Overview of the I/O Stack . . . . .                                                         | 7         |
| 2.2 Top Half: The Interrupt Service Routine (ISR) . . . . .                                     | 7         |
| 2.2.1 Characteristics of the Top Half . . . . .                                                 | 7         |
| 2.3 Bottom Half: Deferred Processing . . . . .                                                  | 8         |
| 2.3.1 Mechanisms Available in Linux . . . . .                                                   | 8         |
| 2.3.2 Why the Split? — The Engineering Rationale . . . . .                                      | 9         |
| 2.4 Summary of Top Half vs. Bottom Half . . . . .                                               | 11        |
| <b>3 The Driver Registration Process</b>                                                        | <b>12</b> |
| 3.1 How a Driver Enters the Kernel . . . . .                                                    | 12        |
| 3.2 <code>module_init</code> and <code>module_exit</code> — The Entry and Exit Points . . . . . | 13        |
| 3.2.1 Pseudocode Walkthrough . . . . .                                                          | 13        |
| 3.3 The LKM Loading Sequence . . . . .                                                          | 16        |
| <b>4 Case Study: Keyboard vs. SSD Driver</b>                                                    | <b>18</b> |
| 4.1 The Two Paradigms of Device I/O . . . . .                                                   | 18        |
| 4.2 The Keyboard Driver — A Character Device . . . . .                                          | 18        |
| 4.2.1 Hardware Interface . . . . .                                                              | 18        |
| 4.2.2 Core Data Structure: <code>file_operations</code> . . . . .                               | 18        |
| 4.2.3 Additional Input-Subsystem Structures . . . . .                                           | 19        |
| 4.3 The SSD Driver — A Block Device . . . . .                                                   | 20        |
| 4.3.1 Hardware Interface . . . . .                                                              | 20        |
| 4.3.2 Core Data Structure: <code>request_queue</code> . . . . .                                 | 20        |
| 4.3.3 The I/O Scheduler’s Role . . . . .                                                        | 21        |
| 4.4 Comprehensive Comparison: Keyboard vs. SSD . . . . .                                        | 22        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Error Handling and Safety</b>                                    | <b>24</b> |
| 5.1      | Why Driver Errors Are Catastrophic in a Monolithic Kernel . . . . . | 24        |
| 5.2      | The Kernel Panic (Linux) . . . . .                                  | 24        |
| 5.2.1    | Common Triggering Conditions . . . . .                              | 24        |
| 5.2.2    | What Happens During a Kernel Panic . . . . .                        | 25        |
| 5.3      | The Blue Screen of Death (Windows) . . . . .                        | 25        |
| 5.4      | The Microkernel Solution: Minix 3 . . . . .                         | 26        |
| 5.4.1    | Architecture Comparison . . . . .                                   | 26        |
| 5.4.2    | The Minix 3 Reincarnation Server . . . . .                          | 27        |
| 5.4.3    | The Trade-off: Performance vs. Reliability . . . . .                | 28        |
| <b>6</b> | <b>Conclusion</b>                                                   | <b>29</b> |
|          | <b>References</b>                                                   | <b>30</b> |

# List of Figures

|     |                                                                                              |    |
|-----|----------------------------------------------------------------------------------------------|----|
| 2.1 | The complete Linux I/O Software Stack — from user application to physical hardware. . . . .  | 7  |
| 2.2 | Detailed flow of the Top Half / Bottom Half interrupt processing model in Linux. . . . .     | 10 |
| 3.1 | LKM loading sequence from <code>insmod</code> to operational device node. . . . .            | 17 |
| 4.1 | Side-by-side data path comparison: keyboard (character) vs. NVMe SSD (block) driver. . . . . | 23 |

# List of Tables

|     |                                                                                       |    |
|-----|---------------------------------------------------------------------------------------|----|
| 2.1 | Comparison of Linux Bottom Half Mechanisms . . . . .                                  | 9  |
| 2.2 | Top Half vs. Bottom Half — Side-by-Side Comparison . . . . .                          | 11 |
| 3.1 | Static Compilation vs. Dynamic LKM Loading . . . . .                                  | 12 |
| 4.1 | Full Comparison of Character (Keyboard) vs. Block (SSD) Driver Architecture . . . . . | 22 |
| 5.1 | Kernel Panic (Linux) vs. BSOD (Windows) — Structural Comparison .                     | 26 |
| 5.2 | Monolithic Kernel vs. Microkernel: Isolation Properties . . . . .                     | 27 |

# Chapter 1

## Introduction

### 1.1 Background and Motivation

---

Modern computers are composed of an eclectic mix of hardware peripherals — keyboards, storage devices, network cards, USB controllers, graphics processors — each manufactured by different vendors, each exposing wildly different electrical and protocol interfaces. Yet every application developer writes the same `read()` and `write()` system calls regardless of whether they are reading from a file on an NVMe SSD or from a USB keyboard. This seamless illusion is the central achievement of the **device driver subsystem** of the Operating System.

A **device driver** is a software module that mediates communication between the OS kernel and a piece of hardware. It translates generic kernel I/O requests into the exact command sequences that the hardware understands, and it converts hardware interrupts and status signals back into the data structures that the kernel expects. Without drivers, every application would need to implement its own hardware-specific code — an engineering impossibility at scale.

### 1.2 Scope of This Report

---

This report targets a firmware engineer who has intimate knowledge of their hardware’s register maps, interrupt lines, and DMA channels, but needs to understand exactly *how* to package that knowledge as a Linux kernel driver. The four sections mirror the four assignment tasks:

1. **Driver Anatomy:** Top Half vs. Bottom Half of interrupt handling.
2. **Registration Process:** Static vs. Dynamic loading; `module_init` / `module_exit`.
3. **Case Study:** Keyboard (character) driver vs. SSD (block) driver — data structures.
4. **Error Handling & Safety:** Kernel Panics, BSODs, and the microkernel remedy.

# Chapter 2

## The Anatomy of a Device Driver

### 2.1 Overview of the I/O Stack

Before diving into interrupt halves, it is important to situate the driver inside the full I/O software stack. Figure 2.1 illustrates the layered architecture from user application to physical hardware.

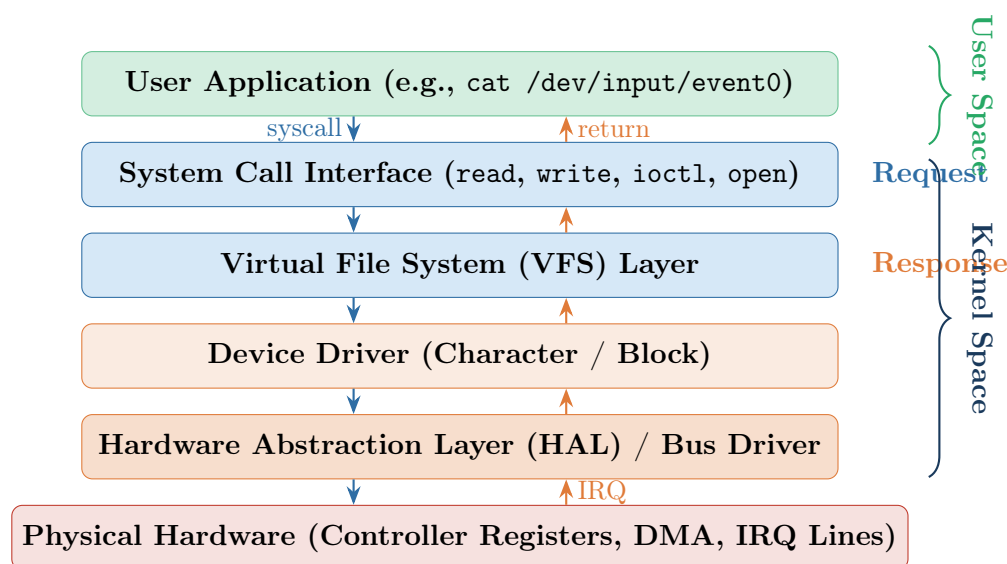


Figure 2.1: The complete Linux I/O Software Stack — from user application to physical hardware.

### 2.2 Top Half: The Interrupt Service Routine (ISR)

When hardware signals the CPU via an **Interrupt Request (IRQ)** line, the processor immediately suspends whatever it was doing and jumps to the address stored in the **Interrupt Descriptor Table (IDT)** for that IRQ number. This entry point is the **Top Half**, also known as the *hard interrupt handler* or *ISR*.

#### 2.2.1 Characteristics of the Top Half

The Top Half executes with interrupts *disabled* on the current CPU. This means:

- **Extremely short execution time** — microseconds at most. Any delay here stalls all interrupt processing on that CPU, increasing latency for every other device in



the system.

- **No sleeping or blocking** — the scheduler cannot preempt the ISR, so calls to `sleep()`, `mutex_lock()`, or any function that might block are strictly forbidden.
- **Minimal work**: acknowledge the hardware interrupt (tell the controller the CPU noticed it), copy time-critical data from hardware registers into a kernel buffer, and then schedule deferred work.

#### What the Top Half Must Do (Mandatory)

1. **Acknowledge the IRQ** to the hardware controller (e.g., write to an EOI register), preventing the device from repeatedly asserting the interrupt line.
2. **Read time-critical hardware registers** (e.g., keystroke scan code, DMA completion status) before the hardware overwrites them.
3. **Schedule the Bottom Half** using one of the kernel's deferred-execution mechanisms (`tasklet`, `workqueue`, or `softirq`).
4. **Return** `IRQ_HANDLED` to the kernel, signalling that this driver owns and processed this interrupt.

## 2.3 Bottom Half: Deferred Processing

The Bottom Half performs the *bulk* of the interrupt processing work — work that can tolerate a small delay and that may require sleeping or locking shared resources.

### 2.3.1 Mechanisms Available in Linux

Linux provides three primary Bottom Half mechanisms, each with distinct trade-offs:

Table 2.1: Comparison of Linux Bottom Half Mechanisms

| Mechanism | Runs In                               | Can Sleep? | Typical Use                              |
|-----------|---------------------------------------|------------|------------------------------------------|
| softirq   | Interrupt context<br>(softirq daemon) | No         | Network Rx/Tx, high-frequency block I/O  |
|           | Softirq context, serialised           | No         | General device drivers (keyboard, mouse) |
| workqueue | Process context<br>(kworker thread)   | Yes        | Disk I/O completion, USB enumeration     |

### 2.3.2 Why the Split? — The Engineering Rationale

The fundamental tension in interrupt handling is between **responsiveness** and **throughput**. If all interrupt processing happened in the ISR (no split), every interrupt would hold off other hardware signals for potentially milliseconds, causing cascading latency. Conversely, if no work happened in the ISR, time-sensitive hardware registers would be overwritten before the OS could read them.

The Top Half / Bottom Half model resolves this by separating the *time-critical minimum* (Top Half) from the *full logical processing* (Bottom Half).

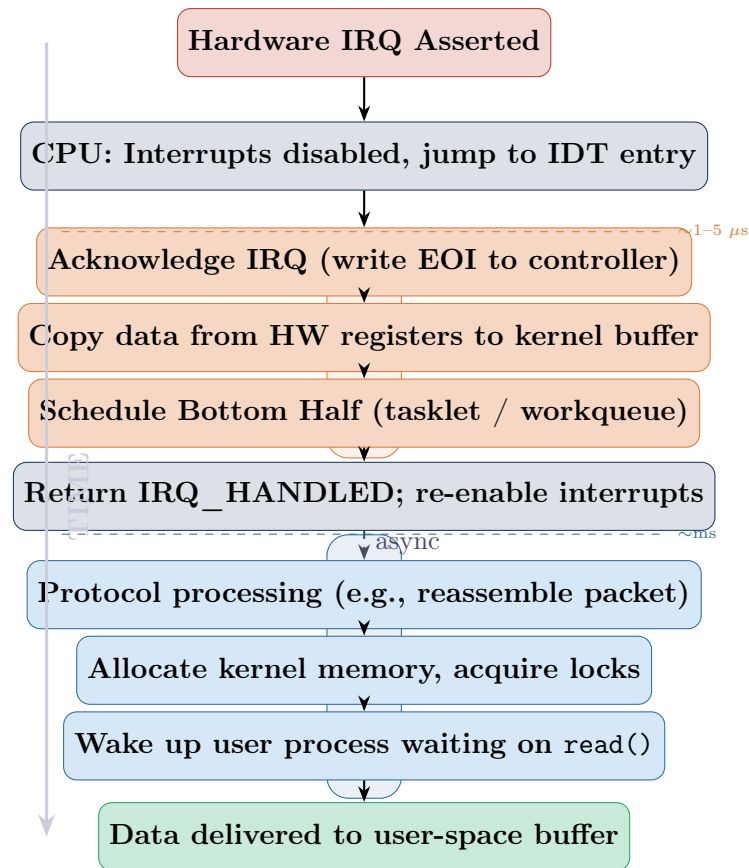


Figure 2.2: Detailed flow of the Top Half / Bottom Half interrupt processing model in Linux.

## 2.4 Summary of Top Half vs. Bottom Half

Table 2.2: Top Half vs. Bottom Half — Side-by-Side Comparison

| Property                    | Top Half (ISR)              | Bottom Half                    |
|-----------------------------|-----------------------------|--------------------------------|
| <b>Interrupts Enabled</b>   | No                          | Yes                            |
| <b>Can Sleep / Block</b>    | Never                       | Yes (workqueue only)           |
| <b>Execution Context</b>    | Hard interrupt context      | Softirq / Process context      |
| <b>Permitted Latency</b>    | $<5\ \mu\text{s}$ typically | Milliseconds acceptable        |
| <b>Scheduling by Kernel</b> | Triggered by hardware IRQ   | Triggered by Top Half          |
| <b>Memory Allocation</b>    | GFP_ATOMIC only             | GFP_KERNEL allowed             |
| <b>Locking Primitives</b>   | Spinlock only               | Mutexes, semaphores, spinlocks |
| <b>Purpose</b>              | Minimal: ack + capture      | Full protocol processing       |

# Chapter 3

## The Driver Registration Process

### 3.1 How a Driver Enters the Kernel

A device driver must *register* itself with the kernel before it can intercept any I/O requests. The kernel maintains tables of registered drivers (indexed by major/minor numbers for character devices, or by request-queue associations for block devices). There are two fundamental ways to load a driver into a running kernel:

Table 3.1: Static Compilation vs. Dynamic LKM Loading

| Attribute             | Static (Built-in)              | Dynamic (LKM)                                            |
|-----------------------|--------------------------------|----------------------------------------------------------|
| Compiled into vmlinux | Yes                            | No                                                       |
| Load Time             | Boot time only                 | Any time ( <code>insmod</code> / <code>modprobe</code> ) |
| Unload without Reboot | Impossible                     | <code>rmmod</code>                                       |
| Kernel Image Size     | Grows                          | Unchanged (loaded on demand)                             |
| Suitable For          | Core platform HW (UART, timer) | Peripheral HW, USB, NIC, SSD                             |
| File Extension        | N/A (baked in)                 | <code>.ko</code> (kernel object)                         |
| Dependency Resolution | Linker at build time           | <code>modprobe</code> at runtime                         |
| Licensing Constraint  | Must be GPL                    | Proprietary possible (TAINT)                             |

### 3.2 `module_init` and `module_exit` — The Entry and Exit Points

Every Linux Kernel Module (LKM) must declare exactly two macro-decorated functions: `module_init()` and `module_exit()`. These are the sole hooks by which the kernel lifecycle manager communicates with the module.

### 3.2.1 Pseudocode Walkthrough

The following pseudocode faithfully represents a complete character-device LKM, annotated to explain every kernel API call.

```

1  /* =====
2  * mydriver.c          Annotated LKM Skeleton (Character Device)
3  * =====
4  */
5  #include <linux/module.h>    /* module_init, module_exit, MODULE_*
6                               macros */
7  #include <linux/kernel.h>    /* printk, KERN_INFO
8                               */
9  #include <linux/fs.h>        /* register_chrdev, file_operations
10                               */
11 #include <linux/cdev.h>      /* cdev_init, cdev_add
12                               */
13 #include <linux/uaccess.h>    /* copy_to_user, copy_from_user
14                               */
15
16 MODULE_LICENSE("GPL");        /* Licence tag (mandatory)
17                               */
18 MODULE_AUTHOR("Your Name");
19 MODULE_DESCRIPTION("Annotated skeleton driver");
20 MODULE_VERSION("1.0");
21
22 /*          Global state
23
24     */
25 static int    major_number;    /* Kernel-assigned major
26     number */
27 static struct cdev  my_cdev;    /* Kernel cdev structure
28     */
29 static struct class *my_class = NULL; /* /sys/class/mydriver
30     */
31 static dev_t  dev_num;        /* Major + minor packed
32     */
33
34 /*          file_operations: the contract with VFS
35
36     */
37 static struct file_operations fops = {
38     .owner    = THIS_MODULE,

```

```

24     .open      = my_driver_open,
25     .release   = my_driver_close,
26     .read      = my_driver_read,
27     .write     = my_driver_write,
28     .unlocked_ioctl = my_driver_ioctl,
29 };
30
31 /* =====
32  * module_init      called by insmod / boot-time linker
33  * =====
34  */
35 static int __init my_driver_init(void)
36 {
37     int ret;
38
39     /* STEP 1: Dynamically allocate a major:minor range.
40      * alloc_chrdev_region() is preferred over register_chrdev()
41      * because it avoids major-number collisions. */
42     ret = alloc_chrdev_region(&dev_num,
43                               0,          /* first minor
44      */
45                               1,          /* count of minors
46      */
47                               "mydriver");
48     if (ret < 0) {
49         printk(KERN_ERR "mydriver: alloc_chrdev_region failed: %d\n", ret);
50         return ret; /* Propagate error; kernel aborts insmod
51      */
52     }
53     major_number = MAJOR(dev_num);
54
55     /* STEP 2: Initialise the cdev structure and link it to fops.
56      */
57     cdev_init(&my_cdev, &fops);
58     my_cdev.owner = THIS_MODULE;
59
60     /* STEP 3: Register cdev with the kernel (makes it live).
61      */
62     ret = cdev_add(&my_cdev, dev_num, 1);
63     if (ret < 0) {
64         unregister_chrdev_region(dev_num, 1);
65         return ret;
66     }
67
68     /* STEP 4: Create a sysfs class so udev auto-creates /dev/
69      mydriver. */

```

```

63     my_class = class_create(THIS_MODULE, "mydriver");
64     if (IS_ERR(my_class)) {
65         cdev_del(&my_cdev);
66         unregister_chrdev_region(dev_num, 1);
67         return PTR_ERR(my_class);
68     }
69
70     /* STEP 5: Create the /dev/mydriver node via udev.
71     */
72     device_create(my_class, NULL, dev_num, NULL, "mydriver");
73
74     /* STEP 6: Register IRQ handler (ISR = Top Half).
75     */
76     ret = request_irq(MY_HW_IRQ,          /* IRQ number from ACPI/
77     DTS      */
78                     my_driver_isr,       /* Top Half handler
79                     */
80                     IRQF_SHARED,        /* Share line if possible
81                     */
82                     "mydriver",
83                     &my_cdev);          /* dev_id for shared IRQ
84
85     check */
86     if (ret) {
87         /* Roll back everything if IRQ cannot be claimed.
88         */
89         device_destroy(my_class, dev_num);
90         class_destroy(my_class);
91         cdev_del(&my_cdev);
92         unregister_chrdev_region(dev_num, 1);
93         return ret;
94     }
95
96     /* STEP 7: Initialise tasklet for Bottom Half.
97     */
98     tasklet_init(&my_tasklet, my_bottom_half, 0UL);
99
100    printk(KERN_INFO "mydriver: loaded, major=%d\n", major_number);
101    return 0;    /* 0 = success. Non-zero aborts loading.
102    */
103 }
104
105 /* =====
106 * module_exit      called by rmmod
107 * =====
108 */
109 static void __exit my_driver_exit(void)
110 {

```



```

100     /* Reverse order of init      critical to avoid use-after-free.
      */
101
102     tasklet_kill(&my_tasklet);           /* Drain pending tasklets
      */
103     free_irq(MY_HW_IRQ, &my_cdev);       /* Release IRQ line
      */
104     device_destroy(my_class, dev_num);    /* Remove /dev/mydriver
      */
105     class_destroy(my_class);             /* Remove /sys/class entry
      */
106     cdev_del(&my_cdev);                  /* Unlink cdev from kernel
      */
107     unregister_chrdev_region(dev_num, 1); /* Free major:minor range
      */
108
109     printk(KERN_INFO "mydriver: unloaded cleanly.\n");
110 }
111
112 /*      Macro registration      tells kernel which functions to
      call      */
113 module_init(my_driver_init);
114 module_exit(my_driver_exit);

```

Listing 3.1: Complete LKM skeleton with `module_init` / `module_exit` (annotated pseudocode)

#### Key Design Principle: Symmetric Teardown

The `module_exit` function must undo *every* resource acquired in `module_init`, in *exactly reverse order*. Failing to release an IRQ, a cdev, or a sysfs class will cause resource leaks that persist until the next reboot — and potentially cause kernel panics if a new driver later attempts to claim the same resource.

### 3.3 The LKM Loading Sequence

Figure 3.1 shows the complete loading sequence from the moment `insmod` is invoked to the point where the device node is accessible.

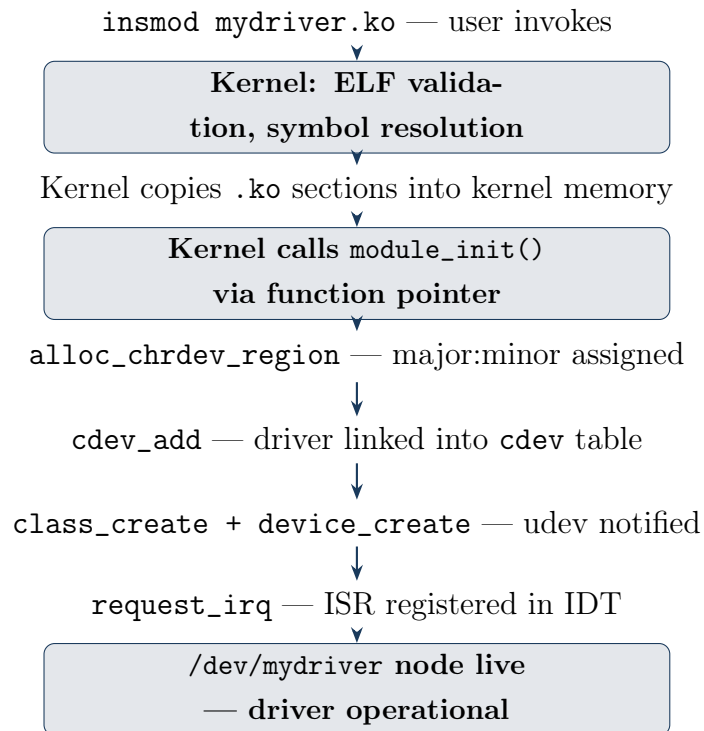


Figure 3.1: LKM loading sequence from `insmod` to operational device node.

# Chapter 4

## Case Study: Keyboard vs. SSD Driver

### 4.1 The Two Paradigms of Device I/O

---

The Linux kernel partitions all devices into two fundamental categories, defined by *how* data flows between hardware and the rest of the system:

- **Character devices** transfer data as a *sequential byte stream* with no inherent block boundaries. There is no kernel-level buffering or reordering — what the hardware sends is what the application receives, one byte (or byte-group) at a time.
- **Block devices** transfer data in fixed-size *blocks* (typically 512 B or 4 KiB). The kernel’s block layer interposes between the driver and the VFS, providing read-ahead caching, I/O scheduling (deadline, CFQ, BFQ, mq-deadline), and request merging.

### 4.2 The Keyboard Driver — A Character Device

---

#### 4.2.1 Hardware Interface

A PS/2 keyboard sends 8-bit *scan codes* to the CPU via IRQ 1 (on x86 systems). Each keypress generates a *make code* and each key release generates a *break code*. The driver’s job is to:

1. Read the scan code from I/O port 0x60 (Top Half).
2. Translate the scan code to a Linux `keycode` (Bottom Half).
3. Place a `struct input_event` on the `input` subsystem’s ring buffer.
4. Wake any process blocking on `read()` from `/dev/input/eventX`.

#### 4.2.2 Core Data Structure: `file_operations`

The central contract between a character driver and the VFS is the `struct file_operations` table. This is a C struct of function pointers — every field is a callback that the VFS invokes when user-space performs the corresponding system call.

```
1  /* From <linux/fs.h>      selected fields */
2  static const struct file_operations kbd_fops = {
3      .owner          = THIS_MODULE,
```

```

4
5     /* Called when user does open("/dev/input/event0", ...)
6     */
7     .open          = kbd_open,
8
9     /* Called on close(fd)      decrements use count, releases
10    buffer */
11    .release        = kbd_release,
12
13    /* Called on read(fd, buf, n)  blocks if no keystrokes
14    pending */
15    .read           = kbd_read,
16
17    /* Keyboard is read-only; write returns -EINVAL
18    */
19    .write           = NULL,
20
21    /* poll/select support      returns POLLIN when keystroke
22    buffered */
23    .poll            = kbd_poll,
24
25    /* ioctl: EVIOCGNAME, EVIOCGID, EVIOCGKEY (key state bitmap)
26    */
27    .unlocked_ioctl = kbd_ioctl,
28
29    /* mmap not meaningful for a byte-stream device
30    */
31    .mmap            = NULL,
32 };

```

Listing 4.1: Keyboard driver file\_operations (annotated)

### 4.2.3 Additional Input-Subsystem Structures

Modern Linux keyboards register with the **Input subsystem** rather than directly with the VFS. The key structure is `struct input_dev`:

```

1 struct input_dev *kbd_dev;
2
3 /* Allocate and populate input_dev */
4 kbd_dev = input_allocate_device();
5 kbd_dev->name      = "My Keyboard";
6 kbd_dev->phys       = "isa0060/serio0/input0";
7 kbd_dev->evbit[0]   = BIT_MASK(EV_KEY);    /* We generate key events
8     */
9 kbd_dev->keybit[...] = /* declare every supported keycode */;

```

```

10  /* Register: from this point, events flow via input_report_key()
    */
11  input_register_device(kbd_dev);

```

Listing 4.2: Keyboard registration via input subsystem

## 4.3 The SSD Driver — A Block Device

### 4.3.1 Hardware Interface

A modern NVMe SSD connects over PCIe and exposes a rich command set (NVMe Express specification). Unlike a keyboard, the SSD:

- Transfers data in sectors (512 B) or logical blocks (4 KiB).
- Supports *out-of-order* command completion — the drive may complete request 5 before request 3 if doing so reduces head-seek latency on an internal flash translation layer.
- Has a *submission queue* and *completion queue* pair per CPU core in NVMe.

### 4.3.2 Core Data Structure: `request_queue`

The kernel block layer manages all I/O through a `struct request_queue`. Each block device has exactly one (or, with NVMe multi-queue, one per CPU), and the block layer deposits `struct request` objects into it.

```

1  /* Allocate the request queue      blk_mq for multi-queue NVMe
    */
2  struct blk_mq_tag_set tag_set = {
3      .ops          = &nvme_mq_ops,      /* callbacks: queue_rq etc.
    */
4      .nr_hw_queues = num_online_cpus(), /* one HW queue per CPU
    */
5      .queue_depth  = 1024,              /* max outstanding commands
    */
6      .cmd_size     = sizeof(struct nvme_cmd_ctx),
7      .flags        = BLK_MQ_F_SHOULD_MERGE, /* allow request merging
    */
8  };
9
10 blk_mq_alloc_tag_set(&tag_set);
11 struct request_queue *q = blk_mq_init_queue(&tag_set);
12
13 /* Attach queue to gendisk (generic disk descriptor)
    */

```

```
14 struct gendisk *disk = alloc_disk(NVME_MINORS);
15 disk->major          = nvme_major;
16 disk->fops            = &nvme_fops;    /* block-level file_operations
    */
17 disk->queue           = q;
18 disk->flags           = GENHD_FL_EXT_DEVT;
19 set_capacity(disk, total_sectors);
20
21 /* Register: /dev/nvme0n1 appears in /sys/block
    */
22 add_disk(disk);
```

Listing 4.3: NVMe/block driver queue initialisation (annotated)

### 4.3.3 The I/O Scheduler’s Role

Once a `struct request` lands in the `request_queue`, the I/O scheduler (e.g., `mq-deadline`) may *merge* it with adjacent requests or *reorder* it for efficiency. This is completely transparent to the driver — the driver only sees the final, optimised batch of requests via its `queue_rq` callback.

## 4.4 Comprehensive Comparison: Keyboard vs. SSD

Table 4.1: Full Comparison of Character (Keyboard) vs. Block (SSD) Driver Architecture

| Aspect                  | Keyboard (Character Device)                                       | NVMe SSD (Block Device)                                         |
|-------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|
| Device node             | <code>/dev/input/eventX</code>                                    | <code>/dev/nvme0n1</code>                                       |
| I/O unit                | Single byte (scan code)                                           | Block / Sector (512 B or 4 KiB)                                 |
| Primary data structure  | <code>struct file_operations</code>                               | <code>struct request_queue</code> + <code>blk_mq_ops</code>     |
| Kernel subsystem        | Input subsystem + chardev                                         | Block layer                                                     |
| I/O scheduling          | None (byte stream)                                                | <code>mq-deadline</code> , <code>BFQ</code> , <code>none</code> |
| Buffer/Cache layer      | Small ring buffer in driver                                       | Page cache (kernel-managed)                                     |
| Data direction          | Unidirectional (HW → CPU)                                         | Bidirectional (READ + WRITE)                                    |
| Interrupt trigger       | Every keypress (IRQ 1 or USB endpoint poll)                       | DMA completion (PCIe MSI-X)                                     |
| Registration API        | <code>register_chrdev</code> / <code>input_register_device</code> | <code>blk_mq_alloc_tag_set</code> + <code>add_disk</code>       |
| Out-of-order completion | Not applicable                                                    | Yes (NVMe CQ per core)                                          |
| Seek optimisation       | N/A                                                               | I/O scheduler handles                                           |
| Typical interrupt rate  | <20 Hz (human typing)                                             | Millions of IOPS                                                |
| DMA involvement         | Rare (some USB HID)                                               | Always (data & PRPs in NVMe)                                    |

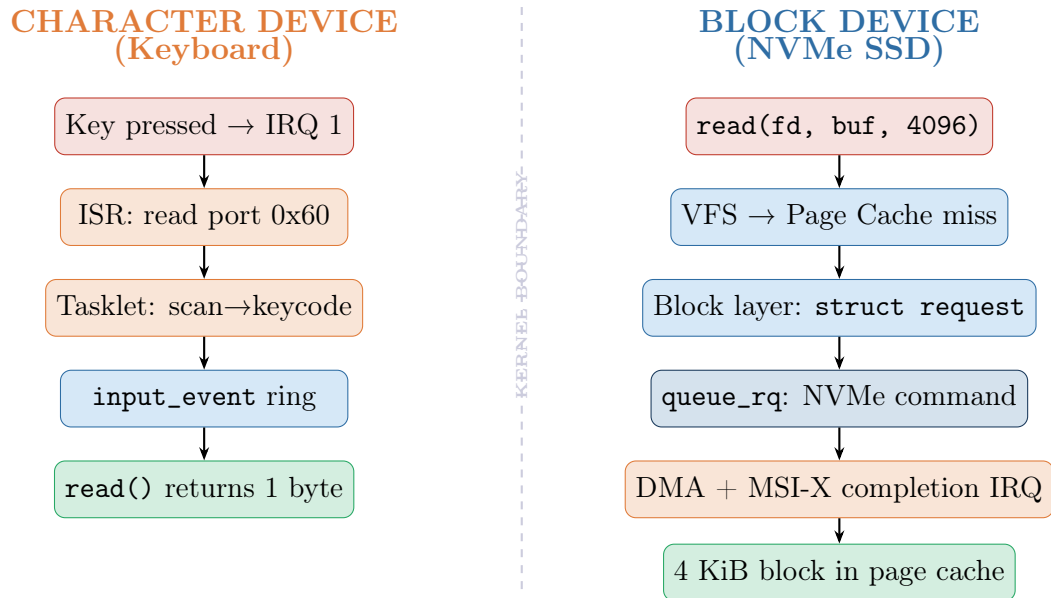


Figure 4.1: Side-by-side data path comparison: keyboard (character) vs. NVMe SSD (block) driver.



# Chapter 5

## Error Handling and Safety

### 5.1 Why Driver Errors Are Catastrophic in a Monolithic Kernel

---

In a **monolithic kernel** (such as mainline Linux), every device driver runs in *Ring 0* — the most privileged CPU execution mode — sharing the same virtual address space as the kernel itself. There is no memory protection between the kernel and its drivers. This architecture delivers exceptional performance but has a profound safety implication: **a bug in a driver is a bug in the kernel**.

### 5.2 The Kernel Panic (Linux)

---

A **Kernel Panic** is the Linux kernel's unrecoverable error handler. It is triggered when the kernel detects a condition from which it cannot safely continue operation.

#### 5.2.1 Common Triggering Conditions

- **NULL pointer dereference in kernel space:** A driver dereferences a pointer that was never initialised (e.g., `dev->buffer->data` where `buffer` is `NULL`). The MMU raises a *page fault* in Ring 0. Since the kernel cannot page-fault to itself, this is unrecoverable.
- **Stack overflow in interrupt context:** A deeply recursive ISR exhausts the small interrupt stack (8 KiB on x86\_64).
- **Double free / use-after-free:** Corrupting the kernel's slab allocator metadata leads to heap corruption that can manifest as arbitrary behaviour, usually detected later as a panic.
- **BUG() / BUG\_ON() assertion:** Driver calls an assertion macro when an impossible condition occurs.
- **Unhandled interrupt in do\_IRQ:** Spurious or unclaimed interrupt floods the IRQ handler loop.

### 5.2.2 What Happens During a Kernel Panic

1. **Interrupts disabled** on all CPUs via `local_irq_disable()`.
2. **Panic message printed** to the kernel ring buffer and serial console: "Kernel panic - not syncing: 0ops[...] RIP: 0010:bad\_driver+0x2a".
3. **Stack trace dumped**: The call chain that led to the fault is logged, invaluable for post-mortem debugging with `addr2line`.
4. **System halted** (or rebooted after `panic_timeout` seconds if configured via `/proc/sys/kernel/pa`).

#### [!] The Kernel Oops vs. Kernel Panic Distinction

A **Kernel Oops** is a *recoverable* error — the kernel logs the fault, kills the offending process, and attempts to continue. A **Kernel Panic** is *unrecoverable*. An Oops in interrupt context automatically escalates to a Panic because there is no process to kill.

## 5.3 The Blue Screen of Death (Windows)

---

Microsoft Windows uses a different but philosophically equivalent mechanism called the **Blue Screen of Death (BSOD)**, technically known as a **Bug Check**. When the Windows Executive detects an unrecoverable error, it calls `KeBugCheckEx(BUGCODE, param1, param2, param3, param4)`.

Table 5.1: Kernel Panic (Linux) vs. BSOD (Windows) — Structural Comparison

| Property                       | Linux Kernel Panic                                             | Windows BSOD (Bug Check)          |
|--------------------------------|----------------------------------------------------------------|-----------------------------------|
| <b>Trigger function</b>        | <code>panic()</code> / <code>BUG()</code>                      | <code>KeBugCheckEx()</code>       |
| <b>Display medium</b>          | Serial console / VGA text mode                                 | Graphical BSOD screen             |
| <b>Error code format</b>       | Symbol + RIP address                                           | 0x0000007E STOP code + params     |
| <b>Memory dump</b>             | <code>/proc/vmcore</code> (kdump)                              | MEMORY.DMP (WinDbg)               |
| <b>Driver isolation</b>        | None in monolithic kernel                                      | None in NT monolithic kernel      |
| <b>Recovery mechanism</b>      | <code>panic_timeout</code> auto-reboot                         | Automatic restart (WER)           |
| <b>Debugging tool</b>          | <code>crash</code> , <code>gdb</code> , <code>addr2line</code> | WinDbg, Microsoft Symbol Server   |
| <b>Driver signing required</b> | Optional (Secure Boot)                                         | Mandatory (WHQL / EV certificate) |

## 5.4 The Microkernel Solution: Minix 3

### 5.4.1 Architecture Comparison

The fundamental reason a driver crash causes a Kernel Panic in Linux is *lack of isolation*. The microkernel design philosophy directly addresses this by moving all non-essential code — including device drivers — out of Ring 0 and into separate user-space processes.

Table 5.2: Monolithic Kernel vs. Microkernel: Isolation Properties

| Property                        | Monolithic (Linux)              | Microkernel (Minix 3)              |
|---------------------------------|---------------------------------|------------------------------------|
| <b>Driver execution level</b>   | Ring 0 (kernel mode)            | Ring 3 (user mode)                 |
| <b>Shared address space</b>     | Yes — driver can corrupt kernel | No — MMU isolates each driver      |
| <b>Driver crash consequence</b> | Kernel Panic / system halt      | Driver process killed; restarted   |
| <b>IPC mechanism</b>            | Direct function call            | Asynchronous message passing       |
| <b>Performance overhead</b>     | Minimal (no context switch)     | IPC latency per I/O request        |
| <b>Fault recovery</b>           | Not possible without reboot     | reincarnation server auto-restarts |
| <b>Kernel size (TCB)</b>        | ~25 million LOC (Linux 6.x)     | ~4,000 LOC (Minix 3 kernel)        |

### 5.4.2 The Minix 3 Reincarnation Server

Minix 3 introduces the **Reincarnation Server (RS)** — a dedicated kernel-level daemon whose sole responsibility is to monitor driver processes. When a driver process crashes (detected via heartbeat timeout or fatal signal), RS:

1. **Kills** the crashed driver process and reclaims its resources.
2. **Forks** a fresh driver process from a stored binary image.
3. **Re-connects** I/O request pipelines to the new process.
4. **Resumes** pending requests — the application may see a brief delay but does not crash.

#### Minix 3 Fault Isolation in Practice

Tanenbaum et al. demonstrated in their 2006 SOSP paper that Minix 3 could survive faults injected into the *every* device driver simultaneously, including disk, network, and TTY drivers, with the system continuing to function after transparent

automatic restart. This level of resilience is structurally impossible in a monolithic kernel without a full reboot.

### 5.4.3 The Trade-off: Performance vs. Reliability

No engineering decision is free of trade-offs. The microkernel’s isolation comes at a performance cost: every I/O system call that crosses from a user-space driver to the microkernel and back requires at least two full *context switches* and two *message-passing* operations. On a modern CPU executing at  $\sim 1$  billion instructions/second, each IPC crossing adds approximately 1–5 microseconds of latency. For a keyboard this is imperceptible; for a high-bandwidth NVMe SSD targeting 7 GB/s throughput, this overhead could be measurable.

Hybrid approaches — such as **exokernels**, **partitioned kernels** (seL4), and Linux’s **eBPF-based driver verification** — attempt to capture safety guarantees without the full IPC penalty.

# Chapter 6

## Conclusion

This report has traversed the full lifecycle of a device driver from a firmware engineer's perspective. The key insights are:

1. **Top Half / Bottom Half split** is not an arbitrary design choice — it is the only viable way to reconcile the sub-microsecond latency requirements of hardware acknowledge cycles with the millisecond-scale processing needed for full protocol handling. Any firmware engineer writing an ISR must keep it brutally short.
2. **Driver registration** is a *contract with the kernel*, not merely an API call. The `module_init` / `module_exit` pair must acquire and release resources symmetrically; any leak is a kernel-level resource leak that cannot be recovered without a reboot.
3. **Character vs. Block** is the most architecturally significant driver decision a firmware engineer will make. Character devices speak byte-stream directly to user-space via `file_operations`. Block devices speak in fixed-size sectors to the kernel's block layer via `request_queue`, benefiting from I/O scheduling, caching, and request merging that the driver does not need to implement.
4. **Safety in a monolithic kernel is fragile by design.** A driver crash is a kernel crash. The microkernel approach (Minix 3) eliminates this coupling through process isolation and automatic driver restart, at the cost of IPC overhead — a trade-off that remains an active area of OS research.

The device driver subsystem is, in the words of one kernel maintainer, "the OS's biggest attack surface and most complex engineering problem." Understanding its architecture deeply is the prerequisite for writing hardware code that the kernel will not merely *tolerate*, but *trust*.

# References

- [1] J. Corbet, A. Rubini, and G. Kroah-Hartman, *Linux Device Drivers*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2005. [Online]. Available: <https://lwn.net/Kernel/LDD3/>
- [2] The Linux Kernel Documentation Project, *The Linux Kernel documentation*, kernel.org. [Online]. Available: <https://www.kernel.org/doc/html/latest/>
- [3] A. S. Tanenbaum, J. Herder, and H. Bos, “Can We Make Operating Systems Reliable and Secure?” *IEEE Computer*, vol. 39, no. 5, pp. 44–51, May 2006.
- [4] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2010.
- [5] B. Ward, *How Linux Works: What Every Superuser Should Know*, 3rd ed. San Francisco, CA: No Starch Press, 2021.
- [6] NVM Express, Inc., *NVM Express Base Specification*, Revision 2.0, 2021. [Online]. Available: <https://nvmexpress.org/specifications/>
- [7] The Linux Kernel, “Block I/O Layer,” *kernel.org Documentation*. [Online]. Available: <https://www.kernel.org/doc/html/latest/block/index.html>
- [8] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2005.